



EXPENSE MANAGEMENT APPLICATION: TECHNOLOGY STACK AND ARCHITECTURE

Team: Debt Solvers

INTRODUCTION

Our team, comprising Alex Zhou, Jonmarc Jackson – Front-end Team, Daniel Chiatuiro – Back-end Team, Hongwei Zhao and Khuong Quan Nguyen – Dev Ops Team, collaborated seamlessly to bring this innovative expense management application to life. Each member contributed their expertise, from leveraging AI technologies to crafting a robust backend using Go and Gin. Together, we designed a scalable, secure, and cloud-native solution that simplifies financial management for users.



1. BUSINESS FOCUS



ALIGNMENT WITH USER NEEDS AND EXPECTATIONS

HOW IT ALIGNS WITH USER NEEDS

- Automates repetitive tasks like expense tracking for efficiency.
- Provides accurate and reliable data extraction for better decision-making.
- Offers real-time insights into spending vs. budgets through visual reports.
- Ensures data privacy and security, addressing users' trust concerns.

WHAT USERS GAIN

- Simplified expense management process.
- Confidence in tracking financial health effortlessly.
- Scalable app experience that grows with their needs.



PROBLEMS

MANUAL DATA ENTRY

Users often face the burden of manually entering expenses, which is time-consuming and prone to errors.

SECURE DATA HANDLING

Protecting sensitive financial information during storage and processing is a critical concern.

INCONSISTENT RECEIPT FORMATS

Different layouts and formats of receipts make it challenging to organize and extract data effectively.

SCALABILITY ISSUES

As the volume of receipts and users grows, traditional systems may struggle to handle the increased demand efficiently.

TRACKING BUDGET VS. EXPENSES

Monitoring spending against a set budget is difficult without a clear and automated system.

SOLUTION

AUTOMATED RECEIPT PARSING

By integrating Azure Custom Vision, Form Recognizer, and the Computer Vision API, the app automates receipt data extraction, reducing manual input and improving accuracy.

REAL-TIME DATA PROCESSING

Leveraging a microservices architecture with the Go and Gin framework ensures low-latency responses and seamless scalability to handle growing user data efficiently.

CATEGORY AND BUDGET MANAGEMENT

Users can create, update, and track custom categories and budgets, helping them organize expenses and align their spending with financial goals.

SECURE DATA STORAGE

Sensitive financial data is protected through JWT-based authentication, secure APIs, and Azure Blob Storage, ensuring user privacy and data integrity.

3 MAIN FEATURES

INVOLVES HANDLING THE EXPENSES

The user is able to go to the category section to view the details and add/delete a budget or expense and it will display the total for the following category. The user may also add or delete categories as well.

INVOLVES AI IMAGE RECOGNITION

The app is able to take a photo of a receipt. If it is clear, it will save it into the default category the user selects as an expense. The user can also upload a receipt from their files.

THE EXPENSE REPORTS AND ANALYTICS

it displays the results of the spending as a pie chart, and the budget as a bar graph. In addition, it also displays the total spending history of the user.

2. TECHNICAL FOCUS



1.CORE SERVICES AND FRAMEWORKS

GO (GOLANG)

The application is built using Go (Golang) for its backend. Go is known for its high performance, scalability, and ease of deployment, making it an ideal choice for developing efficient microservices in this application. It ensures low-latency responses and high concurrency for handling multiple requests, especially crucial for an expense management app that can have frequent data transactions.

GIN FRAMEWORK

For the web framework, Gin is used due to its lightweight and fast nature. It's highly suitable for building RESTful APIs and handles routing, middleware, and request/response management efficiently. It ensures that your app is scalable, maintainable, and easily testable.

MICROSERVICE ARCHITECTURE

The application follows a microservices architecture, where each service (category management, expense tracking, receipt management, budget management) operates independently but communicates through well-defined APIs. This ensures modularity and scalability, allowing future expansion, easier maintenance, and the possibility of scaling services individually as the app grows.

DEBT SOLVERS

2. AI AND MACHINE LEARNING SERVICES (AZURE)

AZURE CUSTOM VISION

Azure Custom Vision is used to train a model specifically designed for receipt data extraction. By feeding the model with clear receipts from various sources (e.g., LCBO, grocery stores), the model learns to identify and extract key information from receipts such as the merchant's name, items, prices, taxes, and dates. This service is fine-tuned for your app's unique requirements, ensuring high accuracy in identifying data relevant to expenses.

CUSTOM VISION USE CASE

The service extracts and parses the receipt and checks if the uploaded image is a valid image based on trained models to identify objects that are receipts and not. If the identified upload image is not a valid receipt image or receipt with a low confidence score, the custom vision returns "Receipt is invalid according to Custom Vision (no positive tag found or too low probability)."

AZURE FORM RECOGNIZER

Form Recognizer is an Azure service that specializes in extracting structured data from scanned documents and forms. This service enhances the receipt processing workflow by extracting data from various receipt formats (e.g., paper, images) and converting it into structured JSON data. It provides an additional layer of flexibility in processing receipts of different layouts, making the app versatile for users submitting receipts from various merchants.

2. AI AND MACHINE LEARNING SERVICES (AZURE)

AZURE COGNITIVE SERVICES (COMPUTER VISION API)

This API is used to extract general text data and visual features from receipt images. It performs OCR (Optical Character Recognition) on receipts to identify and extract text content. While Form Recognizer provides more structured data, the Computer Vision API ensures that the system can recognize text in images, even if the quality or structure is not ideal.

FORM RECOGNIZER USE CASE

Once the receipt is processed through Custom Vision, Form Recognizer can further refine the structured extraction by handling edge cases, such as inconsistent receipt formatting, missing fields, or OCR errors, ensuring a higher accuracy in parsed data.

3. CLOUD AND DATA STORAGE

AZURE STORAGE

Azure Blob Storage is used for storing large image files, including receipts, in a scalable and secure manner. This provides your app with the flexibility to handle large volumes of receipt images without compromising performance.

DATABASE (SQL/NOSQL)

The database backend is a PostgreSQL database (e.g., Azure SQL Database or PostgreSQL) that stores key financial data, including user information, expenses, receipts, and budget allocations. This relational database structure ensures that data is consistent, easily queryable, and maintainable. The app leverages UUIDs as primary keys for all records, ensuring high security and avoiding conflicts in data.

4. API COMMUNICATION & SECURITY

RESTFUL API DESIGN

The system exposes well-defined RESTful APIs to interact with the backend services. Each microservice communicates over HTTP and returns responses in JSON format. The APIs support key operations such as CRUD (Create, Read, Update, Delete) for expenses, receipts, categories, and budgets.

AUTHENTICATION & AUTHORIZATION

The app uses a secure JWT-based authentication system where `**user_id**` is passed with every request to validate the user session. This ensures that sensitive financial data is protected and only accessible to authenticated users.

MIDDLEWARE

Custom middleware is implemented for key tasks such as request logging, rate-limiting, authentication, and error handling.

Middleware in the Gin framework is used to ensure that requests are validated and necessary security checks are applied before interacting with the database.

5. RECEIPT DATA HANDLING

RECEIPT PROCESSING PIPELINE

The receipt processing follows a pipeline:

1. Image Upload: Users upload receipt images.
2. Custom Vision: The image is processed through Custom Vision to extract key data.
3. Form Recognizer: Additional structured data is extracted from receipts with different formats.
4. OCR with Computer Vision API: General text extraction is performed as a backup for unstructured data.
5. Database Storage: The extracted data (items, amount, tax, etc.) is stored in the database for easy access and reporting.

This process minimizes manual input for users and ensures accurate expense tracking.

6. BUDGET AND EXPENSE HANDLING

CATEGORY LOGIC

The app allows users to create, delete or update custom categories in a list to keep track of their own usages. The list of categories also come with default ones that cannot be deleted or modified for the category details. However, the budget and expenses can be adjusted.

MANAGEMENT LOGIC

The functionality allows users to create, update, delete, and track budgets and expenses. The app's architecture ensures that the budget and expense information is stored efficiently and can be adjusted in real-time based on user activity.

ANALYSIS REPORT LOGIC

The budget is linked with the expense entries, so users can see how their actual spending aligns with their planned budgets. A visual report using graphs is displayed based on the user's budget and expense history.

7. SCALABILITY AND FUTURE EXPANSION

MANAGEMENT LOGIC

The use of microservices in the architecture allows for future expansion. Each service (e.g., receipts, categories, expenses) can be scaled independently based on load. This is crucial for handling growth in user data, particularly as the app gains users and processes increasing volumes of receipts.

CLOUD-NATIVE DESIGN

The app is designed to be cloud-native, leveraging Azure's services for storage, machine learning, and cognitive APIs. This ensures that it can scale horizontally, utilizing Azure's infrastructure to handle spikes in demand without impacting performance.

8. FUTURE INTEGRATION AND ENHANCEMENTS

AI-DRIVEN INSIGHTS AND REPORTING

As the app progresses, more advanced AI features can be introduced to provide personalized financial insights, predicting spending patterns, offering savings recommendations, and analyzing long-term financial trends.

THIRD-PARTY INTEGRATIONS

The app can integrate with third-party services like banking APIs or financial institutions, allowing users to sync transaction data directly from their bank accounts.



PRODUCT BENEFITS

Efficiency Through Automation

Real-Time Budget Insights

Scalability and Security

3. LESSONS LEARNED



INSIGHTS GAINED FROM DEVELOPMENT AND TEAMWORK

Throughout the development of the expense management application, we gained valuable insights into the importance of seamless collaboration and clear communication within the team. By leveraging diverse expertise in AI, cloud services, and backend development, we were able to efficiently build and integrate powerful features, such as receipt parsing using Azure's AI tools. Working with a microservices architecture also taught us the importance of modularity and scalability in ensuring long-term flexibility and performance.

1. LESSONS LEARNED

PROBLEM-SOLVING AND ADAPTABILITY

The integration of various AI technologies, like Azure Custom Vision and Form Recognizer, presented challenges in ensuring data accuracy and handling diverse receipt formats. Through iterative testing and refinement, we learned how to fine-tune AI models and improve the reliability of data extraction. Additionally, addressing user security concerns led us to adopt best practices in API design and data storage, ensuring a secure and user-friendly experience.

2. AREAS FOR IMPROVEMENT AND FUTURE PLANS

IMPROVEMENT AREAS

While the app has made significant progress, we identified areas for improvement, particularly in enhancing user interface and experience. Making the app more intuitive and accessible would further help users engage with the features, especially non-tech-savvy individuals. Additionally, further optimizing the AI-driven receipt parsing system to handle more diverse and complex formats could improve overall accuracy.

FUTURE PLANS

Moving forward, we plan to incorporate more advanced AI features to provide users with predictive financial insights and savings recommendations. We also aim to expand the app's integration with external services, such as banking APIs, to automate the categorization of transactions and enable real-time updates. Furthermore, we'll continue to enhance the app's scalability and security as we expand its user base globally.

SUMMARY

Your expense management application leverages cutting-edge AI technologies like Azure Custom Vision, Form Recognizer, and Computer Vision API for seamless receipt parsing and data extraction, coupled with the robust and scalable Go and Gin framework for the backend. The cloud-native architecture ensures that the application is scalable, secure, and capable of handling growth, making it an excellent choice for future development and deployment. With the power of machine learning, cloud storage, and a microservices-based design, this app is poised to revolutionize how users manage and track their finances effortlessly.



THANK YOU

Debt Solvers

Seneca Polytechnic, Toronto